

Day 4 总结：aster-drm 组件骨架创建

日期: 2026-04-10 | 分支: drm | commit: 9b19e872

Day 4 总结：aster-drm 组件骨架创建

日期: 2026-04-10 分支: drm commit: 9b19e872

今日交付物

文件	说明
kernel/comps/drm/ Cargo.toml	组件包配置，依赖 aster-framebuffer、 ostd、spin
kernel/comps/drm/src/ lib.rs	组件入口，#[init_component] 宏 初始化
kernel/comps/drm/src/ buffer.rs	BackBuffer 双缓冲数据结构
kernel/comps/drm/src/ simple.rs	SimpleDrm 驱动协调层 + present()
Components.toml	注册 drm = { name = "aster- drm" }

架构讲解

三层模块关系

lib.rs (入口)

- 导出 BackBuffer, SimpleDrm

- init() 空实现 (Phase 1 不需要初始化资源)

buffer.rs (数据层)

- BackBuffer { data: Vec<u8>, width, height, bpp, line_size }

- 提供: data_mut(), clear(), write_pixel_at(), write_bytes_at()

- 生命周期: 程序运行期间一直存在

simple.rs (逻辑层)

- SimpleDrm { fb, back_buffer }

- new() - 从 FRAMEBUFFER 拿硬件信息, 创建 BackBuffer

- back_buffer() - 返回 Mutex guard, 可直接操作像素

- present() - 原子性: 整屏 memcpy 到硬件

SimpleDrm 内部结构



BackBuffer 字段含义

字段	来源	含义
<code>data:</code> <code>Vec<u8></code>	<code>Vec::new(width * height * bpp)</code>	整屏像素容器，在内存中
<code>width</code>	<code>fb.width()</code>	屏幕宽（像素）
<code>height</code>	<code>fb.height()</code>	屏幕高（像素）
<code>bpp</code>	<code>fb.pixel_format().nbytes()</code>	每像素字节数 (2=RGB565, 3=RGB888, 4=BGRX8888)
<code>line_size</code>	<code>fb.line_size()</code>	每行字节数（可能 > width * bpp，对齐）

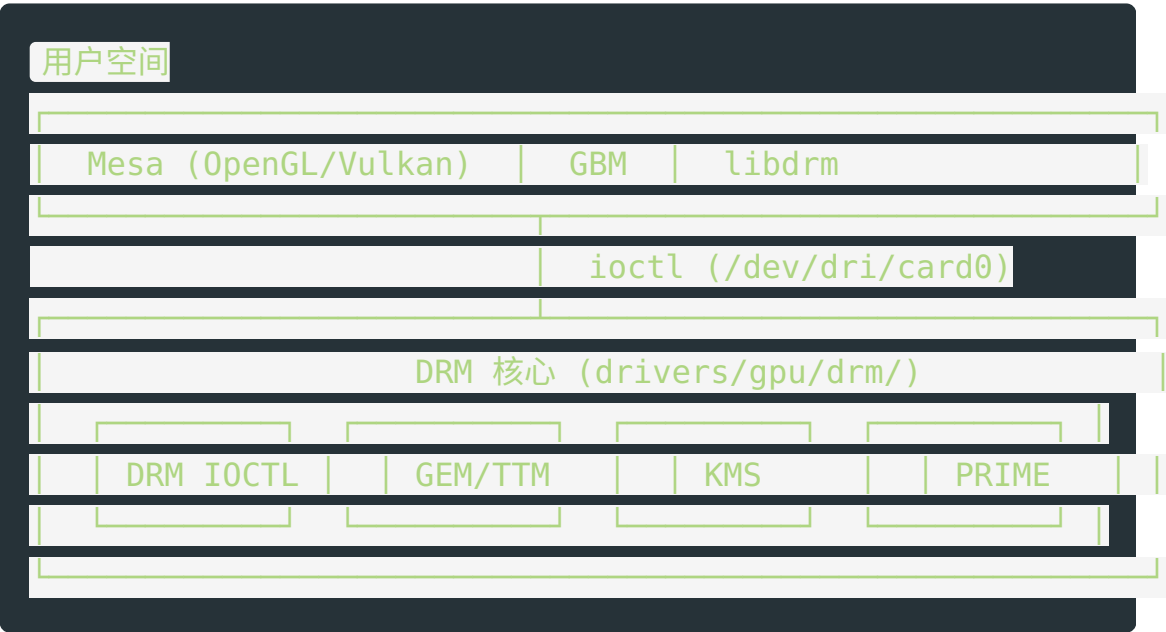
Phase 1 vs Linux DRM

核心差异：内存位置

维度	Linux DRM/KMS	SimpleDrm Phase 1
内存位置	VRAM（GPU 专用显存）	RAM（系统内存）
内存管理	GEM/TTM 复杂管理	<code>Vec<u8></code>
模式设置	KMS 动态切换分辨率	无（bootloader 固定）
设备节点	<code>/dev/dri/card0</code>	无
垂直同步	DRM 页面翻转	无，盲目整屏 memcpy

SimpleDrm 本质是"单缓冲 + 整屏同步刷新"，不是真正的双缓冲。真正的页面翻转（page flip）需要 Phase 3+。

Linux DRM 完整架构（参考）



测试方式

为什么不用 C 程序测？



Phase 1 使用**内核侧测试**：在 `lib.rs` 的 `init()` 里直接调用 `SimpleDrm`，画棋盘格，然后 `present()`。内核启动时自动执行，无需用户态程序。

下一步 (Day 5-6)

1. 在 `lib.rs` 的 `init()` 中添加棋盘格测试逻辑
 2. `make kernel` 编译
 3. `make run_kernel` 运行, 验证屏幕上显示图案
 4. 如需要调整像素格式或偏移量, 修改 `buffer.rs` 中的写入逻辑
-

关键 API 参考

```
// 创建 SimpleDrm (从 FRAMEBUFFER 全局变量拿硬件信息)
let drm = SimpleDrm::new()?;

// 获取背缓冲 (返回 Mutex guard)
let mut buf = drm.back_buffer();

// 画棋盘格示例
buf.clear(0x000000); // 全黑
// ... 按像素写入 ...

// 刷新到屏幕 (全屏 memcpy)
drm.present()?;
```